Beyond Memory and Retrieval: A Taxonomy of Interrupted Agentic Work Grounded in Coding-Agent Resumption

Anonymous Author[a]

[a] *Independent Research,*

ARTICLE INFO

Keywords:
agent continuity
interrupted work
coding agents
resumption
authority
work state

ABSTRACT

When an autonomous agent resumes previously interrupted work, the central failure is often not that relevant information is missing. It is that the agent cannot determine which visible state is entitled to govern resumed action. We call this the authority problem and argue that it is not adequately captured by memory persistence, retrieval relevance, workflow visibility, or durable goal structure alone.

We propose a taxonomy of interrupted agentic work organized around four primary objects: current work, authority, readiness, and next valid action, with authority as the pivotal object that links visible records to justified resumed action. On top of these objects, we distinguish interruption classes, loss classes, a dominant-assignment rule, and auxiliary recovery states.

The taxonomy is grounded in coding-agent resumption, where the problem is sharpest: environment drift is rapid, side effects are often irreversible, and multiple heterogeneous records routinely compete for governing status at the resumption boundary. We argue, however, that the core objects, especially authority and readiness, name a broader structural gap in current agent-continuity thinking that extends beyond software engineering.

The contribution is taxonomic rather than mechanistic. It defines a missing analytical layer for agent resumption and derives design requirements for future continuity systems, including explicit authority semantics, readiness modes, and pre-action verification mechanisms. The taxonomy is constructed so that its classes can be tested through structured episode classification and inter-annotator agreement. The central claim is that future agent systems should optimize not only for carrying more context forward, but for recovering the right work state under minimal inherited session state.

1. Introduction

1.1. The Resumption Problem in Long-Horizon Agent Systems

Autonomous agents increasingly operate across extended time horizons: multi-session coding tasks, long-running research workflows, iterative planning cycles, and multi-step tool-mediated operations. As these systems grow in capability, they also grow in vulnerability to a class of failures that has received surprisingly little systematic attention: failures at the resumption boundary.

Consider a coding agent that returns to a repository after a session pause. The visible active context still points to work X. The issue thread is still open. A rich note explains the intended next edit. But a later structured state record shows that work X was blocked pending approval and that work Y now governs. Both states are relevant. Only one of them should decide what happens next. If the agent keeps implementing work X, it has not failed because it lacked context. It has failed because it trusted the wrong visible state.

This example names the failure mode this paper is about. It arises not from information absence but from unresolved precedence among competing persisted records at the moment of resumed action. We call this the authority problem. The example is drawn from coding-agent work, but the structural pattern — multiple plausible state records surviving an interruption boundary, with no explicit rule to settle which one governs — recurs wherever agents operate across sessions, handoffs, or asynchronous state changes.

We ground the taxonomy in coding work because that is where the problem is sharpest. In software environments, environment drift is rapid, side effects are often hard to reverse, and continuation pressure can push the system into irreversible action under the wrong governing record. But the objects we identify

ORCID(s):

— authority, readiness, next valid action — are not intrinsically limited to software. They name a structural gap in how current agent systems reason about resumed work more broadly.

1.2. Why This Is Not Only a Memory Problem

At first glance, interrupted agent work looks like a memory problem. The prior plan, the latest notes, the changed files, or the recent conversation may no longer fit in the active context window, so the obvious response is to preserve more context, summarize better, or retrieve more effectively later. That response is often useful, but it is incomplete.

Contemporary agent-memory systems have made exactly this persistence question serious, whether through hierarchical long-lived memory, reflective summaries, architectural memory decomposition, or explicit memory layers [1–5]. These systems represent genuine progress. But in the work X case, both states were present. The blocker record was there. The active context was there. Memory was not the problem.

This is why the protagonist of the paper is not memory but authority. A stale note and a fresh structured blocker can both be remembered. A summary and a later status transition can both be retrieved. The hard part is deciding which one is entitled to govern the resumed decision. Once authority is unresolved, readiness often becomes unresolved with it: the system no longer knows whether it should act, ask, wait, escalate, or abstain.

This distinction becomes sharper as agents are used across multiple sessions or long-running harnesses. A design optimized only for carried-over context treats continuity as a problem of retaining more of the prior conversation. But accumulated conversational residue is not the same as governing work state. It can be stale, superseded, or structurally misaligned with the resumptive decision the system now faces. Recent systems work on long-horizon agents and KV-cache management reinforces the same lesson: carrying ever more context forward is not free [6, 7]. Long-lived context creates memory pressure, operational cost, and more room for stale state to remain silently influential. The stronger target for continuity is therefore reliable recovery under limited inherited session state, not unlimited retention.

1.3. Why Current Frames Are Incomplete

The role of the next section is to map five near-misses. Each neighboring literature gets close to the resumption problem from a different direction, but each stops one layer too early. That residue matters especially in coding work, where environment drift is rapid and continuation pressure can push the system into irreversible action under the wrong governing record, but the gap also extends to any domain where agents must adjudicate among competing persisted records before acting.

1.4. Methodological Grounding

The taxonomy was refined through iterative prototype development, structured failure analysis across multiple resumptive cases, and repeated classification exercises that exposed recurring patterns not cleanly captured by adjacent framings. Those repeated pressures are one reason authority and readiness became the load-bearing objects of the present account.

The present article is not the mechanism paper, but neither is it offered as a purely speculative viewpoint. The taxonomy is constructed so that its classes support structured validation through authored interrupted-work episodes classified independently by human annotators, with an auxiliary LLM annotation tier as a secondary consistency check. The goal is to define the failure surface and the missing objects precisely enough that later mechanism papers can test runtime interventions directly.

1.5. Thesis and Contributions

The thesis of this paper is that interrupted agent work — grounded here in coding-agent resumption — is not adequately explained by memory, retrieval, workflow visibility, or durable goal structure alone. It requires a work-state-centered taxonomy of failure organized around current work, authority, readiness, and next valid action, with record-level entitlement as a domain-specific operationalization of source precedence that turns visible state into justified resumed action.

The paper makes three contributions. First, it defines interrupted coding work as a distinct failure surface and proposes a taxonomy spanning interruption classes, loss classes, dominant assignment, and auxiliary recovery states. Second, it identifies record-level entitlement over competing persisted records as

an under-operationalized object in current agent-resumption accounts and uses the taxonomy to show why persistence, relevance, workflow visibility, and durable goal structure each remain insufficient on their own. Third, it derives design requirements and a research agenda for future continuity systems, including explicit authority semantics, explicit readiness modes, and stronger pre-action verification at resumptive boundaries.

2. Adjacent Frames and Their Limits

This section is organized around five near-misses. Each adjacent frame captures a real part of interrupted agent work, and each helps explain why the problem is hard. But each one stops one layer too early in a different way. The point of the section is not to dismiss neighboring literatures. It is to show why authority at the resumed decision boundary remains analytically underexposed.

2.1. Task Interruption and Resumption

The interruption and resumption literature correctly sees that resumed work fails before any downstream task-solving can even begin. Foundational work by Trafton and colleagues and by Altmann and Trafton shows that interruption cost depends on suspended goals, resumptive cues, and the conditions under which a prior task can be re-entered efficiently [8, 9]. Matthews and colleagues extend this by showing that interface-level information design can improve reacquisition [10]. Iqbal and Horvitz further demonstrate that disruption and recovery are recurring design problems in real computing environments [11, 12].

Within software engineering specifically, Parnin and Rugaber and DeLine and Parnin show that interrupted programming tasks are difficult to resume and that external cues and supports materially affect reacquisition quality [13, 14]. This legitimizes the basic problem: resumption is a long-standing property of complex work, not a synthetic inconvenience of modern agent tooling.

What this literature does not yet provide is a machine-facing account of governing state. It was developed for human operators and for cue design in human-facing environments. It does not formalize machine-readable work state, source entitlement, action readiness, or the distinction between resumed work identity and resumed action validity for autonomous agents. The implication is that good resumptive cues are necessary but not sufficient. A continuity system must additionally represent which work currently governs and what record is entitled to define its present state.

2.2. Agent Memory and Long-Lived LLM State

Memory work is the most important modern near-neighbor because it has already elevated persistence into a serious design object. Systems such as MemGPT, Reflexion, and Mem0, together with broader architectural accounts such as CoALA and recent memory surveys, ask how agents should preserve, organize, retrieve, and reuse state across extended interaction [1–5].

This literature has made several important contributions. It has clarified that memory can be represented in multiple forms: summaries, reflections, compressed traces, salient facts, episodic buffers, and external stores. CoALA in particular distinguishes working memory, episodic memory, semantic memory, and procedural memory, providing a structured decomposition that moves well beyond treating persistence as a monolithic problem [3]. MemGPT introduces tiered memory management that includes operations resembling precedence decisions across memory layers [1].

The fit is still incomplete because memory work usually asks what information should survive and become reusable later. The persistent unit is typically a memory item, a reflection, a summary, or a useful fact. Our paper asks a narrower and different question: what governs one interrupted piece of work at the moment of resumption? That question cannot be answered by persistence alone, because the system may remember several plausible artifacts without knowing which one is currently entitled to govern action.

In the work X case, a memory system that preserved both the active context and the blocker record would still leave the central question unanswered. Both persisted records survive. The system still does not know which one governs. The gap is not that memory is irrelevant. The gap is that persistent memory does not by itself resolve the governing state of interrupted work. The design implication is that persistent memory has to be subordinated to a governing work-state layer that decides which preserved record is currently entitled to govern one interrupted work item.

2.3. Agent Workflows and Persistent Task Context

Workflow and agent-system papers get a different part of the story right: complex agentic work is procedural, iterative, and full of intermediate state. ReAct, MetaGPT, SWE-agent, AutoCodeRover, traceability-oriented studies, and vision pieces on intelligent development environments all contribute to a picture in which agent performance depends on structured interfaces, iterative workflows, tool-mediated action, and visible intermediate state [15–20].

These papers help explain why the problem is timely. Once agents participate in iterative work, interruptions, handoffs, validation failures, and cross-session resumptions become normal rather than exceptional. Workflow systems show that agents benefit from explicit structure and that complex work is increasingly distributed across artifacts, tools, and actors.

But workflow visibility still leaves an analytical gap. A visible task board, trace, or state snapshot does not automatically determine which work is currently primary, which source governs resumed action, or whether the next valid move is action or non-action. Workflow systems help make state visible; they do not yet isolate the governing resumptive state as a separate object.

The implication is that workflow-aware systems need an explicit resumptive layer above visibility. Making traces and state legible is useful, but interrupted work still requires a representation that decides which visible state outranks the others and whether action is valid now.

2.4. Classical Practical Reasoning, Intentions, and Commitment

Classical practical-reasoning traditions are a serious near-neighbor, especially BDI-style work on beliefs, desires, intentions, commitment, and intention reconsideration. This tradition already formalizes several issues that resemble parts of the present taxonomy: durable goals, commitment persistence, reconsideration under changing beliefs, and conditions under which action should or should not proceed [21–24]. Any account of interrupted agent behavior that ignores this tradition risks overstating its own novelty.

The overlap is real, especially on the readiness side of the taxonomy. A BDI framing predicts that interrupted agents should succeed once they preserve the current intention clearly enough and reconsider it when beliefs change. That framing is strong as far as it goes. Schut et al. [24] on intention reconsideration provide trigger conditions that determine when belief change should force reconsideration, a mechanism that is functionally adjacent to parts of the authority problem studied here.

But the work X case fails on a different dimension. Belief revision says how an agent should update what it takes to be true. The authority problem here asks which visible record is entitled to cause that update for one interrupted work item in the first place. Authority is best understood as a domain-specific operationalization of a broader source-precedence problem that practical-reasoning traditions treat more abstractly. BDI architectures illuminate reconsideration and commitment, but they are not primarily operationalized around notes, trackers, logs, workspace cues, superseding state transitions, and other heterogeneous persisted records whose precedence must be settled before resumed action can be justified.

The contribution here is therefore narrower than a new general theory of practical reasoning. It is a claim that artifact-rich interrupted work — grounded here in coding but potentially generalizable — places unusual pressure on record-level precedence, and that this pressure is not yet operationalized cleanly by memory, retrieval, workflow, or internal intention structure alone.

The implication is not that BDI-style structure becomes irrelevant, but that interrupted agent systems operating in artifact-rich environments need one more layer between belief revision and action: an explicit account of which persisted record gets to drive belief update and readiness assessment for the current work item.

2.5. Retrieval-Oriented Context Shaping

Retrieval work is the sharpest foil because it comes closest to saying, "this is just a better context-selection problem." Foundational RAG work, in-context retrieval augmentation, robustness studies, and code-domain retrieval systems all show that the shape and quality of retrieved context materially affect downstream reasoning [25–29]. More recent context-engineering work for long-horizon agents strengthens this further by showing that context growth, compression, and selection are now central design problems.

Our answer is that retrieval relevance and governing work state are not the same object. In the work X case, both records are relevant. That is exactly why retrieval alone cannot settle the problem. A record may

be highly relevant to the work and still be the wrong basis for resumed action if a later blocker, superseding transition, or stronger state record overrides it. The design implication is that retrieval for interrupted work cannot optimize only for relevance. It also needs a way to rank or reject records by governing entitlement.

2.6. What Remains Missing Across These Frames

Across these adjacent strands, a common limitation remains. Each literature captures part of the interrupted-work problem, but none treats resumptive work state as the primary object of analysis. Task-interruption studies explain why resumptive cues matter but do not model machine-readable work state. Agent-memory work externalizes long-lived state but usually treats the persistent unit as a memory item rather than as the governing state of one unfinished work item. Retrieval approaches improve context selection, but relevance is not the same as authority. Workflow systems show that agentic work is iterative, but they generally optimize task execution rather than interruption-boundary state resolution. Practical-reasoning traditions formalize durable intention and commitment revision, but they are not primarily framed around artifact precedence across heterogeneous persisted records.

What remains missing is a model of interrupted agentic work in which four objects are first-class: current work, authority, readiness, and next valid action. Among these, authority is the sharpest gap: the central failure is often not that an agent cannot remember enough context, but that it cannot determine which persisted record is entitled to govern action.

3. The Failure Surface of Interrupted Coding Work

The work X case fails because four questions break at once. Which work now governs? Which persisted record is entitled to decide that? Is action currently valid? And if so, what is the next justified step? Those questions are separable, and resumed work fails in different ways depending on which one breaks first. A taxonomy should therefore organize itself around them.

We ground the taxonomy in coding-agent work because that is where we have the deepest developmental experience and where the structural pressures are most acute. Section 6 returns to the question of how far these objects generalize.

3.1. Unit of Analysis and Scope

The unit of analysis is one *resumptive decision episode*: a bounded episode in which an agent must determine, after interruption or handoff, what work is currently governing, what state is authoritative for that work, whether action is currently permitted, and what the next valid action should be. This unit is narrower than the full life of a task and narrower than general long-term memory. It focuses on the decision boundary immediately before resumed action, because that is where interrupted work repeatedly fails despite apparently adequate context.

3.2. Core Objects

3.2.1. Work State

Work state is the compact governing state of one work item at the interruption boundary. It should be strong enough to answer what the work currently is, whether it is active, blocked, waiting, superseded, or done, and what kind of next move that status permits. In the work X case, the failure is not that the repository lacks information. It is that no compact governing state has won cleanly enough to decide whether X is still active or whether Y now governs.

Treating work state as first-class distinguishes the present framing from both document retrieval and conversational memory. In a retrieval framing, the task is to surface relevant evidence. In a work-state framing, the task is to determine which work item is current and what state transition, if any, it currently permits.

3.2.2. Authority

Authority is the answer to the question that defines the work X failure: which persisted record gets to govern the resumed decision? A record is authoritative when it can override competing signals for the same work state, reflects the latest accepted state transition for that work, and does not itself depend on unresolved reconciliation with a stronger record.

This notion is central because interruption-boundary failures often occur after the relevant information is already present. The system sees a note, a log, an active context, a tracker status, or a workspace artifact, but cannot decide which of those is entitled to govern the next step. The claim is not that provenance or source conflict is unknown in neighboring literatures, but that authority has not been centered as a first-class object in current agent accounts of interrupted work recovery.

3.2.3. Readiness

Readiness asks a simpler but more dangerous question: even if the system knows what work governs, is action currently valid at all? The canonical readiness modes are `act`, `ask`, `wait`, `escalate`, and `abstain`. Many resumptive failures are not failures of choosing the wrong concrete step; they are failures of attempting action when the correct current outcome is non-acting.

This distinction matters because systems optimized for continuation pressure will systematically overproduce `act`. A blocked work item may still be correctly identified, yet the correct next move may be to wait, ask, escalate, or abstain. In coding work, that bias is especially expensive because action can create irreversible side effects. But the same continuation-pressure bias exists in any domain where agents are rewarded for producing output rather than for producing justified output.

3.2.4. Next Valid Action

Next valid action is the smallest admissible next step consistent with the current work state, the authoritative evidence, and the current readiness mode. We divide it into *action mode* (whether to act, ask, wait, escalate, or abstain) and *action content* (what concrete step to take once the mode is fixed). This separation allows the taxonomy to distinguish `readiness_loss` from `intent_loss`: the former concerns uncertainty about the correct mode, while the latter concerns uncertainty about the concrete step after the mode is already fixed.

3.3. Interruption Classes

Interruption classes describe the kind of boundary event that produces a resumptive decision episode. They name the boundary condition, not the failure itself. The same interruption class may damage different objects in different cases.

The interruption classes proposed here are session cutoff, task switch, blocked waiting, environment drift, failure boundary, handoff, multi-open-work conflict, false done, and dirty done. Together they cover the main ways interrupted coding work becomes resumptively difficult without claiming to exhaust every possible interruption. The list is not intended as a final exhaustive ontology; other candidates, such as review-driven rejection, gradual context-window overflow, or dependency cascades, may deserve separate treatment in later revisions.

Session cutoff denotes episodes where a prior working session ends before local trajectory is fully externalized. Task switch denotes episodes where priority ordering is redirected across works. Blocked waiting captures interruptions in which the work remains identifiable but action permissibility depends on missing input, approval, or an unresolved dependency. Environment drift captures cases where the live environment has changed enough that previously recorded state may no longer be trustworthy. Failure boundary marks episodes where the previous session ended at a failed command or validation step. Handoff denotes transfer across model or operator boundary where tacit rationale is lost even if durable artifacts remain.

The remaining three classes capture ambiguity that often gets flattened in generic workflow systems. Multi-open-work conflict refers to episodes where several open works remain simultaneously plausible as current. False done denotes episodes where the system resumes against a completion-looking state even though closure conditions were never actually met. Dirty done denotes episodes where a valid completion marker exists, but unresolved follow-up obligations mean the work still governs future action.

Table 1 summarizes the interruption classes.

3.4. Loss Classes

Loss classes identify the primary object that the resumptive episode cannot currently resolve. They describe the location of the failure in the resumptive decision itself.

The five loss classes are focus loss, authority loss, readiness loss, intent loss, and closure loss. Focus loss applies when current-work identity itself is unresolved. Authority loss applies when a plausible work

Table 1
Interruption classes.

Interruption class	Operational definition	Primary object pressured	Typical wrong move
session cutoff	the prior working session ends before state is fully externalized	loss of immediate plan continuity	resume the correct work from the wrong local step
task switch	attention is redirected to another work item or priority changes	loss of current-work focus	resume the most recent-looking work instead of the true priority
blocked waiting	the work remains identifiable but action depends on missing input, approval, or dependency	readiness uncertainty	continue acting instead of waiting, asking, or escalating
environment drift	the live environment has changed enough that prior state may be untrustworthy	authority degradation	act on state that has become stale
failure boundary	the prior session ended at an execution, tooling, or validation failure	readiness and recovery ambiguity	retry blindly or reopen too much context
handoff	work passes across model, operator, or session boundary and tacit state is lost	loss of tacit rationale and intent	misread the same artifacts and diverge from prior trajectory
multi-open-work conflict	several open works remain simultaneously plausible as current	focus and authority competition	pick the richest artifact trail instead of the truly current work
false done	a closure-looking assertion appears even though closure evidence was never satisfied	closure ambiguity	close too early and remove still-active work
dirty done	a valid completion marker exists alongside residual obligations that still govern action	closure ambiguity	suppress still-governing follow-up work

Table 2
Loss classes.

Loss class	Governing question that fails	Inclusion rule	Exclusion rule	Assignment order
focus loss	Which work is truly current?	use when current-work identity is unresolved	do not use if the work is known but action is unclear	1
authority loss	Why should this state be trusted over competing signals?	use when the problem is adjudicating between sources or validating freshness	do not use if the trusted state is known and the dispute is only about action mode or content	2
readiness loss	Which action mode is currently valid?	use when the main uncertainty is whether action is currently permitted	do not use if the action mode is already determined	3
intent loss	What concrete next step should be taken?	use when the action mode is known but action content is unresolved	do not use if the main uncertainty is whether the agent should act at all	4
closure loss	Is this work actually done or still active?	use when closure state is the main unresolved object	do not use if the work remains clearly active	5

state is visible but the system cannot determine why one state source should govern over competing signals. Readiness loss applies when work and trusted state are known but the correct action mode remains unresolved. Intent loss applies once readiness is resolved and only the concrete next step remains unclear. Closure loss applies when the work's completion status is the main unresolved object.

This ordering is deliberate. If the current work is unknown, there is no stable basis for downstream reasoning. If work is known but authority is unresolved, the system may act on the wrong governing truth. If both are fixed but readiness is unresolved, even a plausible step may be invalid now. Only after these conditions are satisfied does it make sense to ask about concrete action content.

Table 2 defines the loss classes operationally.

Table 3
Auxiliary recovery states.

Recovery state	Meaning	Typical entry condition	Exit condition
unresolved	no safe current-work candidate exists yet	the episode enters recovery before any governing work candidate is stable	a candidate work is identified
candidate found	a work candidate exists but still needs authority or freshness validation	a plausible current work is visible but precedence is unresolved	authority and freshness are sufficiently resolved
waiting on one fact	one specific fact blocks safe resolution	one missing fact prevents safe classification or action	the fact is obtained or the system abstains
blocked	the work is known but the valid action mode is not act	approval, dependency, or external input is required	the blocker clears or the system escalates
stale	previously recorded state is no longer trustworthy	environment drift or superseding evidence makes prior state suspect	live state is reverified or action is withheld
ready	current work and next valid action are both sufficiently justified	work, authority, and readiness have been resolved	action is taken or passed onward
done pending close	substantive work appears complete but lifecycle closure is unresolved	implementation appears complete while closure obligations remain	the work is closed cleanly or reopened

3.5. Dominant-Assignment Rule

Many resumptive episodes contain several plausible problems at once. To keep the taxonomy usable, each episode receives one *dominant loss class*: the first missing or invalid object whose resolution is necessary before a safe next valid action can be produced. This rule does not deny that multiple pressures may coexist. Its purpose is to prevent classification from collapsing into an undisciplined list of everything relevant.

The dominant-assignment order follows the dependency structure of safe resumption: focus → authority → readiness → intent → closure. A higher-priority class dominates only when the corresponding governing object must be resolved before any safe downstream inference is possible.

3.6. Auxiliary Recovery States

A secondary recovery-state layer shows how a continuity system may move through an interruption-boundary episode once recovery begins. This layer is auxiliary rather than load-bearing.

3.7. Representative Examples

Representative examples help show that the taxonomy is not tied to one narrow implementation. The cases below are phrased generically, although they are informed by the developmental history behind this work. They should be read as illustrative application rather than as claim-bearing empirical validation.

In the work X episode, the visible session context still points to work X, but a later structured state record shows that work X was blocked and that work Y now governs. The agent resumes work X because the visible context is richer. This is a multi-open-work conflict case; the dominant loss is authority loss, because the failure lies in selecting the wrong governing record rather than in total ignorance of plausible candidates.

In a "known work, invalid action" episode, the correct work is obvious and the governing record is trusted, but the work is blocked pending approval. The agent continues implementation. This is a blocked-waiting interruption whose dominant loss is readiness loss: the system fails at action mode rather than at work identity or record selection.

In an "intent-loss after valid mode" episode, the current work is known, the authoritative state is trusted, and the correct action mode is clearly `act`, but two concrete next steps remain plausible. The agent guesses without resolving the underlying detail. This is an intent-loss case: the action mode is already fixed and only concrete action content remains underdetermined.

A contrastive "stateless agent" example shows that the taxonomy does not depend on any one continuity system. An agent re-enters work by scanning only recent diffs, issue text, and chat history. Two unfinished goals remain plausible, and the agent picks the one with richer textual evidence. The dominant loss is focus loss: the system cannot reliably determine which work is actually current.

An "over-structured tracker" case shows that failure can arise from the wrong state source winning, not only from too little state. A tracker marks a work item as `in progress`, while a more recent engineering-side transition shows the work is blocked. The agent trusts the tracker and keeps executing. The dominant loss is authority loss; only after the governing source is corrected would the blocked non-action mode become visible.

4. Design Implications

4.1. Work as the Primary Continuity Object

Future continuity systems should treat `work`, not only conversation or artifact recall, as the primary continuity object. This means representing current-work identity, current status, and minimal admissible next-move constraints directly rather than inferring them from residue spread across notes, traces, and messages. This requirement is consistent with older coordination research on shared work understanding as well as with newer agent-system directions that externalize persistent state [19, 30–33].

4.2. Explicit Authority Semantics

Future systems should expose explicit `authority semantics` rather than leaving source precedence implicit inside retrieval scores or workflow views. In practical terms, the system should be able to answer why one persisted record outranks another for the same work item and under what conditions that ordering should be reconsidered. Explicit authority semantics belong to the representation layer, not only to downstream prompting.

4.3. Explicit Readiness Modes and Non-Action Outputs

Future systems should represent `readiness` explicitly instead of assuming that successful continuity culminates in immediate action. The correct present outcome may be to ask, wait, escalate, or abstain. Systems that collapse all successful recovery into `act` will continue to mishandle exactly the cases where resumed action is most dangerous. Non-action outcomes — `waiting on approval`, `ask for one missing fact`, `revalidate stale state`, `abstain pending authority resolution` — must become first-class outputs, not failures of helpfulness.

4.4. Pre-Action Verification

The taxonomy points toward a requirement that future systems verify governing state before resumed action rather than assuming that reconstructed context is already trustworthy. Emerging long-horizon agent work already points toward bounded-context reconstruction and plan-aware context management [33, 34]. If the taxonomy is correct, then systems should not only recover candidate work state but also check for divergence between the state they are about to trust and the state that the artifact field now actually justifies.

5. Discussion and Research Agenda

5.1. Implications for Agent Evaluation

Coding-agent benchmarks should not measure only whether the model eventually produces a plausible patch. They should also measure whether the model recovers the right current work, resolves authority correctly, chooses the right readiness mode, and selects the next valid action under interruption pressure. Current evaluation regimes often reward eventual action quality under whatever context was supplied [35–41]. A continuity-sensitive evaluation regime should additionally reward correct abstention, escalation, waiting, and source-sensitive re-entry.

The taxonomy is designed so that its classes can be tested through structured episode classification rather than through system-comparison benchmarks alone. In such a design, interrupted-work episodes are authored from public traces, provenance-visible historical artifacts, and deliberately constructed boundary cases, then classified independently by human annotators. Interruption-class agreement, dominant-loss agreement, and disagreement clusters can then reveal whether the taxonomy is independently usable and where its boundaries need revision.

5.2. Implications for Runtime Continuity Systems

A strong continuity system should be judged not only by how much prior context it can carry forward, but by how well it can reconstruct governable work state under minimal inherited session state. Runtime systems may converge on smaller inherited contexts and stronger state externalization for practical reasons [6, 7, 33, 34]. The taxonomy adds a more specific claim: restartability matters not only because long context is expensive, but because resumed correctness depends on the explicit reconstruction of work, authority, readiness, and next valid action.

5.3. Beyond Coding: Toward a General Account of Interrupted Agentic Work

The taxonomy is grounded in coding-agent work because that is where the problem is most acute and where the developmental evidence is strongest. But the core objects — authority, readiness, next valid action — are not intrinsically limited to software.

Several structural features make coding work an especially sharp grounding domain. Environment drift is rapid because repositories, dependencies, and CI states change continuously. Side effects are often hard to reverse because committed code, merged branches, and deployed artifacts create durable consequences. The artifact field is unusually heterogeneous because issue trackers, pull requests, commit histories, chat threads, configuration files, and workspace state all coexist as competing records. And continuation pressure is strong because coding agents are typically optimized to produce patches, commits, or resolutions.

But analogous pressures exist in other domains of extended agentic operation:

- Research agents operating across multi-session literature review, experimental design, and manuscript preparation face similar authority problems when prior notes, evolving hypotheses, and newer experimental results compete at session boundaries.
- Planning and scheduling agents managing long-horizon task graphs encounter resumption failures when task dependencies shift, priorities change, or external constraints evolve between sessions.
- Autonomous operations agents monitoring infrastructure, managing deployments, or coordinating multi-step workflows face environment drift and record precedence problems structurally similar to those in coding work.
- Multi-agent systems in which different agents contribute to shared artifacts face authority problems at handoff boundaries, where one agent's state record may conflict with another's.

The hypothesis is that authority, readiness, and next valid action are general objects of interrupted agentic work, not only of coding. Coding work is where the pressure is easiest to see and where the developmental evidence is richest, but the analytical gap may extend to any setting where agents must adjudicate among heterogeneous persisted records before acting. Testing that hypothesis across domains is an open research question.

5.4. Open Research Questions

Several research questions remain open. First, how stable are the proposed class boundaries once broader datasets or independent annotators are introduced? Second, how domain-specific is this taxonomy? Some elements may generalize broadly, but the exact role of environment drift, irreversible side effects, and multi-artifact precedence may still be unusually strong in software settings.

Third, what is the right operational test for authority resolution? A future system will need some explicit way to justify why one record outranks another. Fourth, how should evaluation distinguish successful re-entry from superficially plausible continuation? And finally, what is the smallest continuity surface that still preserves high-quality recovery under near-zero session inheritance?

5.5. From Taxonomy to Mechanism

This paper is intentionally not the mechanism paper. Even so, the taxonomy points toward one concrete next step: systems should test for divergence before acting on resumed state. If resumed work fails when the wrong persisted record governs action, then a mechanism should exist that checks whether the candidate governing state has been superseded, invalidated, or blocked before execution proceeds. A later paper can ask whether explicit pre-action divergence checks reduce authority-sensitive resumed-action failures in practice.

6. Limitations

The taxonomy remains partly author-derived. It was assembled through literature synthesis, repeated structured failure analysis, and direct interaction with one family of continuity artifacts. The class boundaries may partly reflect the authors' framing choices rather than an already settled ontology.

External validation remains limited. The taxonomy is constructed so that its classes support structured validation through authored episodes and independent classification, but the present manuscript reports the analytical contribution rather than a completed empirical study. Readers who expect a systems contribution should read this accordingly: the claim is taxonomic, not mechanistic.

Some boundary cases may remain unstable. Authority-loss and focus-loss cases can interact closely when closure failure hides the true current work. Readiness-loss and intent-loss cases can look deceptively similar when a system is uncertain about both mode and content. And the boundary between interrupted coding work and interrupted agentic work more generally remains open.

7. Conclusion

Interrupted agentic work is easy to misdescribe as a generic memory or retrieval problem. The argument of this paper is that something more specific breaks at the resumed decision boundary. The system must recover not only relevant context, but the right current work, the right governing record, the right action mode, and the next valid action that follows from them.

That is why the paper centers authority rather than treating it as a minor side condition. In many resumed failures, the decisive problem is not that the relevant traces are absent. It is that several traces survive and the system cannot tell which one is entitled to govern action.

We ground the taxonomy in coding-agent work because that is where the problem is sharpest, the evidence is richest, and the cost of wrong resumption is most visible. But the objects we identify — authority, readiness, next valid action — name a broader structural gap in how current agent systems reason about resumed work. If this analysis is right, future continuity systems should optimize not only for memory persistence or better retrieval, but for clean recovery of governable work state under minimal inherited session state.

That shift — from "carry more forward" to "recover the right governing state" — may be the most consequential design implication of the present analysis, not only for coding agents, but for any agent system that must resume interrupted work reliably.

References

[1] Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez, MemGPT: Towards LLMs as Operating Systems, arXiv preprint arXiv:2310.08560, 2023.

[2] Noah Shinn, Federico Cassano, Edward Berman, and Ashwin Gopinath, Reflexion: Language Agents with Verbal Reinforcement Learning, arXiv preprint arXiv:2303.11366, 2023.

[3] Theodore R. Sumers, Shunyu Yao, Karthik Narasimhan, and Thomas L. Griffiths, Cognitive Architectures for Language Agents, arXiv preprint arXiv:2309.02427, 2023.

[4] Zeyu Zhang et al., A Survey on the Memory Mechanism of Large Language Model based Agents, arXiv preprint arXiv:2404.13501, 2024.

[5] Prateek Chhikara et al., Mem0: Building Production-Ready AI Agents with Scalable Long-Term Memory, arXiv preprint arXiv:2504.19413, 2025.

[6] Junliang Wang et al., Multi-tier Dynamic Storage of KV Cache for LLM Inference Under Resource-Constrained Conditions, Complex & Intelligent Systems, 2026.

[7] Yusen Wu et al., KVC-Q: A High-Fidelity and Dynamic KV Cache Quantization Framework for Long-Context Large Language Models, Journal of Systems Architecture, 2026.

[8] J. Gregory Trafton, Erik M. Altmann, Derek P. Brock, and Farilee E. Mintz, Preparing to Resume an Interrupted Task: Effects of Prospective Goal Encoding and Retrospective Rehearsal, International Journal of Human-Computer Studies, 2003.

[9] Erik M. Altmann and J. Gregory Trafton, Task Interruption: Resumption Lag and the Role of Cues, Proceedings of the 26th Annual Conference of the Cognitive Science Society, 2004.

[10] Tara Matthews et al., Clipping Lists and Change Borders: Improving Multitasking Efficiency with Peripheral Information Design, CHI 2006.

[11] Shamsi T. Iqbal and Eric Horvitz, Disruption and Recovery of Computing Tasks: Field Study, Analysis, and Directions, CHI 2007.

[12] Shamsi T. Iqbal and Eric Horvitz, Conversations Amidst Computing: A Study of Interruptions and Recovery of Task Activity, User Modeling 2007.
[13] Chris Parnin and Spencer Rugaber, Resumption Strategies for Interrupted Programming Tasks, ICPC 2009.
[14] Robert DeLine and Chris Parnin, Evaluating Cues for Resuming Interrupted Programming Tasks, CHI 2010.
[15] Shunyu Yao et al., ReAct: Synergizing Reasoning and Acting in Language Models, ICLR 2023.
[16] Sirui Hong et al., MetaGPT: Meta Programming for A Multi-Agent Collaborative Framework, arXiv preprint arXiv:2308.00352, 2023.
[17] John Yang et al., SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering, NeurIPS 2024.
[18] Tianlin Yang et al., AutoCodeRover: Autonomous Program Improvement, arXiv preprint arXiv:2404.05427, 2024.
[19] Mark Marron, A New Generation of Intelligent Development Environments, IDE Workshop, arXiv:2406.09577, 2024.
[20] Ira Ceka et al., Understanding Software Engineering Agents Through the Lens of Traceability: An Empirical Study, arXiv preprint arXiv:2506.08311, 2025.
[21] Michael E. Bratman, Intention, Plans, and Practical Reason, Harvard University Press, 1987.
[22] Philip R. Cohen and Hector J. Levesque, Intention Is Choice with Commitment, Artificial Intelligence, 1990.
[23] Anand S. Rao and Michael P. Georgeff, Modeling Rational Agents within a BDI-Architecture, KR 1991.
[24] Martijn Schut, Michael Wooldridge, and Simon Parsons, The Theory and Practice of Intention Reconsideration, Journal of Experimental and Theoretical Artificial Intelligence, 2004.
[25] Patrick Lewis et al., Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks, arXiv preprint arXiv:2005.11401, 2020.
[26] Ori Ram et al., In-Context Retrieval-Augmented Language Models, arXiv preprint arXiv:2302.00083, 2023.
[27] Ori Yoran et al., Making Retrieval-Augmented Language Models Robust to Irrelevant Context, arXiv preprint arXiv:2310.01558, 2023.
[28] Fengji Zhang et al., RepoCoder: Repository-Level Code Completion Through Iterative Retrieval and Generation, arXiv preprint arXiv:2303.12570, 2023.
[29] Disha Shrivastava et al., RepoFusion: Training Code Models to Understand Your Repository, arXiv preprint arXiv:2306.10998, 2023.
[30] Alberto Espinosa et al., Shared Mental Models and Coordination in Large-Scale, Distributed Software Development, ICIS, 2001.
[31] Prasun Dewan and Rajesh Hegde, Semi-Synchronous Conflict Detection and Resolution in Asynchronous Software Development, ECSCW 2007.
[32] Sharon M. Ryan and Rory O'Connor, Acquiring and Sharing Tacit Knowledge in Software Development Teams: An Empirical Study, Information and Software Technology, 2013.
[33] Chenglin Yu et al., InfiAgent: An Infinite-Horizon Framework for General-Purpose Autonomous Agents, arXiv preprint arXiv:2601.03204, 2026.
[34] Kamer Ali Yuksel, PAACE: A Plan-Aware Automated Agent Context Engineering Framework, arXiv preprint arXiv:2512.16970, 2025.
[35] Yuchen Zhuang et al., ToolQA: A Dataset for LLM Question Answering with External Tools, NeurIPS Datasets and Benchmarks Track, 2023.
[36] Xiao Liu et al., AgentBench: Evaluating LLMs as Agents, arXiv preprint arXiv:2308.03688, 2023.
[37] Carlos E. Jimenez et al., SWE-bench: Can Language Models Resolve Real-World GitHub Issues?, arXiv preprint arXiv:2310.06770, 2023.
[38] Xuhui Xiao et al., τ-bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains, arXiv preprint arXiv:2406.12045, 2024.
[39] Adyasha Maharana et al., Evaluating Very Long-Term Conversational Memory of LLM Agents, arXiv preprint arXiv:2402.17753, 2024.
[40] Yilun Liu et al., LongMemEval: Benchmarking Chat Assistants on Long-Term Interactive Memory, arXiv preprint arXiv:2407.11017, 2024.
[41] Jian Zhang et al., SWE-bench Goes Live!, arXiv preprint arXiv:2504.13008, 2025.